

Time Series Analysis

Havana Rika



Ben-Gurion University
of the Negev

Time-Series Forecasting

Classical Forecasting for Time-Series

- **Forecasting** future values from historical observations
- **Evaluation metrics**

Topics Covered

- ❖ Moving Average (MA)
- ❖ Autoregressive (AR) Models
- ❖ ARMA and ARIMA
- ❖ Exponential Smoothing
- ❖ ARCH and GARCH
- ❖ Vector Autoregression (VAR)

Classical Time-Series Models

Classical forecasting models are:

- Based on **statistical** theory
- **Widely used** in econometrics
- Designed primarily for **univariate** forecasting
- **Effective** when data is limited
- Often assume **stationarity**

Examples: **MA /AR/ARMA /ARIMA/VAR / GARCH**

Stationarity TS

Most classical time-series models are **linear** and assume **stationarity** (constant statistical properties over time).

They perform well when data is limited, but often require preprocessing such as detrending or differencing to enforce stationarity.

Real-world time-series are frequently **non-linear and non-stationary**, making them difficult to model with simple statistical approaches.

While classical models offer simplicity and interpretability, modern forecasting increasingly relies on **machine learning and neural networks**, which can capture more complex patterns.

Forecasting Workflow

Typical Forecasting Process

1. Test for stationarity
2. Differencing [if stationarity detected]
3. Fit method and forecast
4. Add back the trend and seasonality

Moving Average (MA)

Core Idea

- Forecast future values using previous random shocks (errors).
- Simple Moving Average:
 - Average of previous observations
 - Smooths short-term fluctuations
 - Reveals long-term trends

Benefits:

- Simple
- Easy to interpret
- Effective for smoothing

$$\frac{x_1 + x_2 + \dots + x_k}{k} = \frac{1}{k} \sum_{i=0}^k x_i$$

Simple moving AVG

Moving Average (MA)

```
from src.forecasting.baselines import NaiveMovingAverage
#Taking a moving average over 48 timesteps, i.e, one day
naive_model = NaiveMovingAverage(window=48)
```

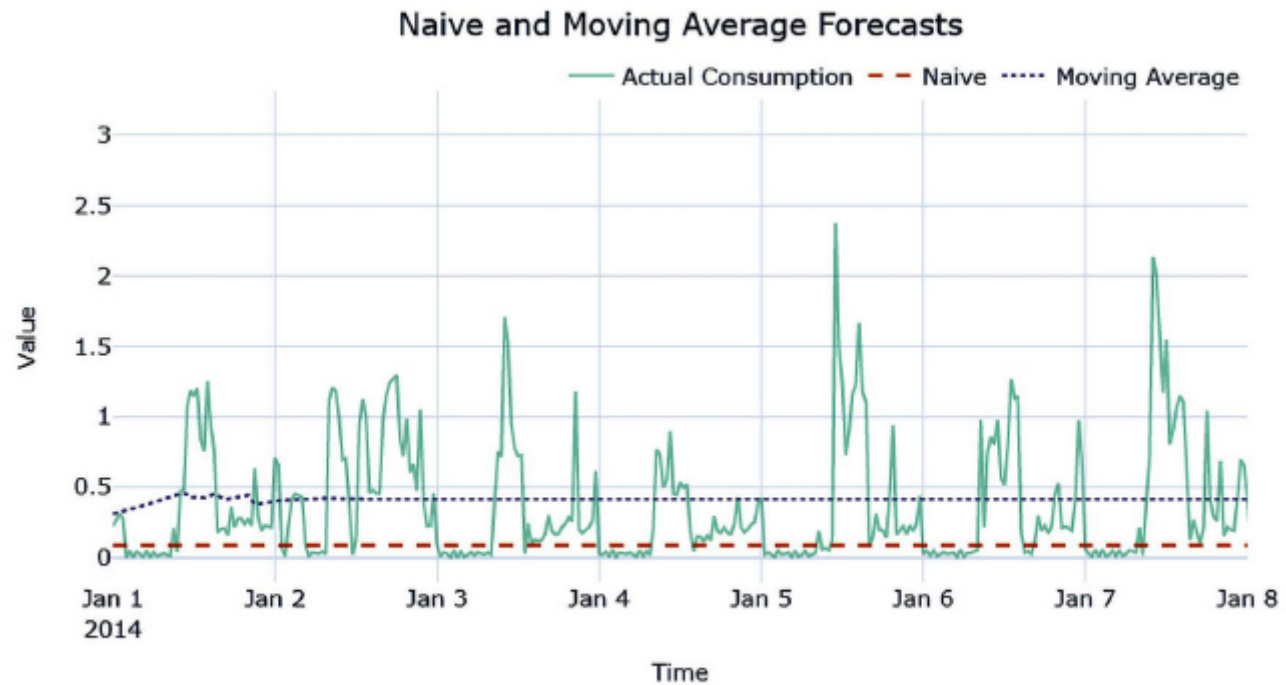


Figure 4.2: Moving average forecast

MA(q) Model

Moving Average Model of order q

$$x_t = \mu + \epsilon_t + \sum_{i=1}^q \phi_i \epsilon_{t-i}$$

Where:

μ = the average (expectation) of x_t (usually assumed to be 0)

q = number of lagged errors

ϕ_i = weights given to the past error terms

ϵ_{t-i} = previous white noise

ϵ_t = random noise

The model predicts current values using past forecast errors.

MA(q) Model

What does MA(q) actually model?

Notice that the predictors are:

$$\epsilon_{t-1}, \epsilon_{t-2}, \dots$$

These are residuals (innovations), not previous observations.

So an MA(q) process can be viewed as:

A stationary series generated by a finite memory of past shocks.

For example,

$$x_t = \mu + \epsilon_t + 0.7\epsilon_{t-1}$$

means that a shock today influences the series for one additional time step.

AR Models Intuition

Core Idea

Current values depend on previous observations.

Example

Predict today's stock price using:

- Yesterday's price
- Price two days ago
- Price three days ago

AR models capture **temporal dependencies** directly from **past values**.

Wold's Decomposition

In 1938, Norwegian mathematician Herman Ole Andreas Wold described the decomposition of stationary time-series.

He observed that **stationary** time-series can be expressed as the sum of a **deterministic component** (autoregressive) and a **stochastic component** (noise).

This decomposition is termed after him today, as **Wold's decomposition**.

Autoregressive (AR) Models

This leads to the following formulation of the **autoregressive model** of order p , $AR(p)$, as:

$$x_t = c + \sum_{i=1}^p \phi_i x_{t-i} + \epsilon_t$$

Where:

p = number of lags

c = constant

ϕ_i = autoregressive coefficients whights

Autoregressive (AR) Models

- Apply the AR model to the **original** time series (or differenced series) as the primary **forecasting** engine.
- Apply the AR model to the **residuals** to **correct errors**, **remove** remaining **autocorrelation**, or **improve** the **accuracy** of a base forecast.

Autoregressive Moving Average (ARMA)

Combining AR and MA

ARMA merges:

AR component → past observations

MA component → past errors

$$ARMA(p, q)$$

Advantages:

More flexible than AR or MA alone

Captures multiple dependency structures

Look at the autocorrelation and partial autocorrelation plots, where we could see peaks in the correlation for each lag to set p and q .

ARMA Equation

$$x_t = c + \epsilon_t + \sum_{i=1}^p \phi_i x_{t-i} + \sum_{i=1}^q \theta_i \epsilon_{t-i}$$

Combines:

- Autoregressive terms
- Moving average terms

→ Requires stationary data.

ARIMA Models

Handling Non-Stationary Data

ARIMA(p, d, q) is an autoregressive integrated moving average model, which includes a data **preprocessing** step, called integration, to **make** the time-series **stationary**, which is achieved by subtracting the immediate past values, a transformation called differencing.

Parameters:

p = autoregressive order

d = differencing order

q = moving average order

The most widely used classical forecasting model.

Differencing

Differencing is a transformation used in time-series analysis to remove trends and make a series more **stationary**.

Instead of modeling the values themselves, we model the **changes between consecutive values**.

Integration Step

Transform:

$$x_t - x_{t-1}$$

Purpose:

- Remove trends
- Stabilize mean
- Achieve stationarity

The parameter d specifies how many times differencing is applied.

ARIMA Models

```
from statsforecast.models import (ARIMA, AutoARIMA)

#ARIMA model by specifying parameters
arima_model = ARIMA(order = (2,1,1), seasonal_order = (1,1,1), season_length =
48)

#AutoARIMA model by specifying max limits for parameters and letting the
algorithm find the best ones
auto_arima_model = AutoARIMA( max_p = 2, max_d=1, max_q = 2, max_P=2, max_D =
1, max_Q = 2, stepwise = True, season_length=48)
```

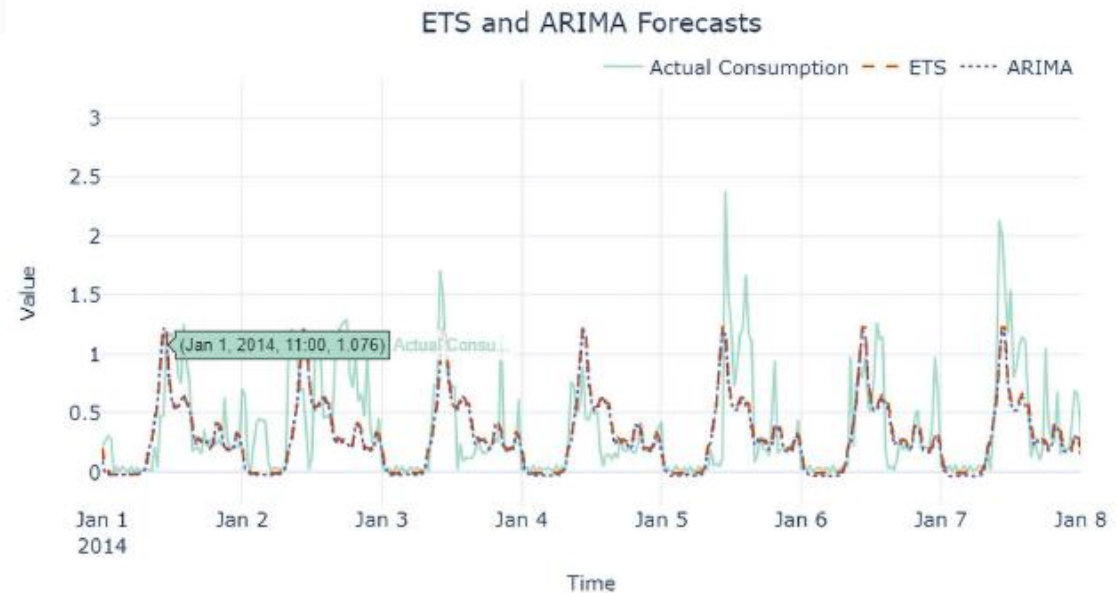


Figure 4.5: ETS and ARIMA forecasts

Seasonal Naive Forecast

```
from statsforecast.models import SeasonalNaive  
seasonal_naive = SeasonalNaive(season_length=48*7)
```

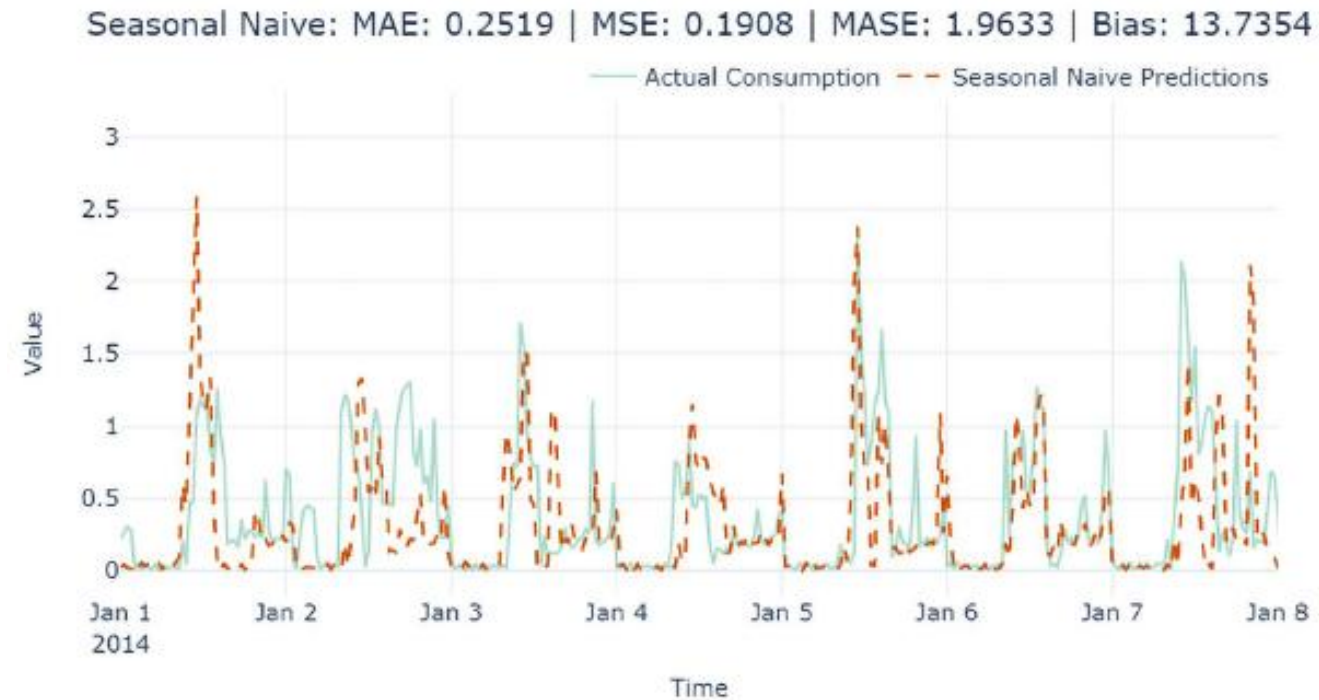


Figure 4.3: Seasonal naive forecast

Seasonal ARIMA (SARIMA)

Modeling Seasonality

$$ARIMA(p, d, q)(P, D, Q)_m$$

Additional seasonal parameters:

- m denotes the number of periods in a season
- P, D, Q parametrize the autoregressive, integration, and moving average components of the seasonal part
- p, d, q refer to the ARIMA terms, which we've discussed previously

Suitable for yearly, monthly, weekly patterns.

Seasonal ARIMA (SARIMA)

Seasonal Autoregression, SAR

$$SAR(P): x_t = c + \sum_{i=1}^P \phi_i x_{t-s \cdot P} + \epsilon_t,$$

Seasonal moving average, SMA

$$SMA(Q): x_t = \mu + \epsilon_t + \sum_{i=0}^Q \varphi_i \epsilon_{t-s \cdot Q},$$

Each of these components will use a distinct set of parameters.

Seasonal ARIMA (SARIMA)

```
from statsforecast.models import (ARIMA, AutoARIMA)

#ARIMA model by specifying parameters
arima_model = ARIMA(order = (2,1,1), seasonal_order = (1,1,1), season_length = 48)

#AutoARIMA model by specifying max limits for parameters and letting the algorithm find the best ones
auto_arima_model = AutoARIMA( max_p = 2, max_d=1, max_q = 2, max_P=2, max_D = 1, max_Q = 2, stepwise = True, season_length=48)
```

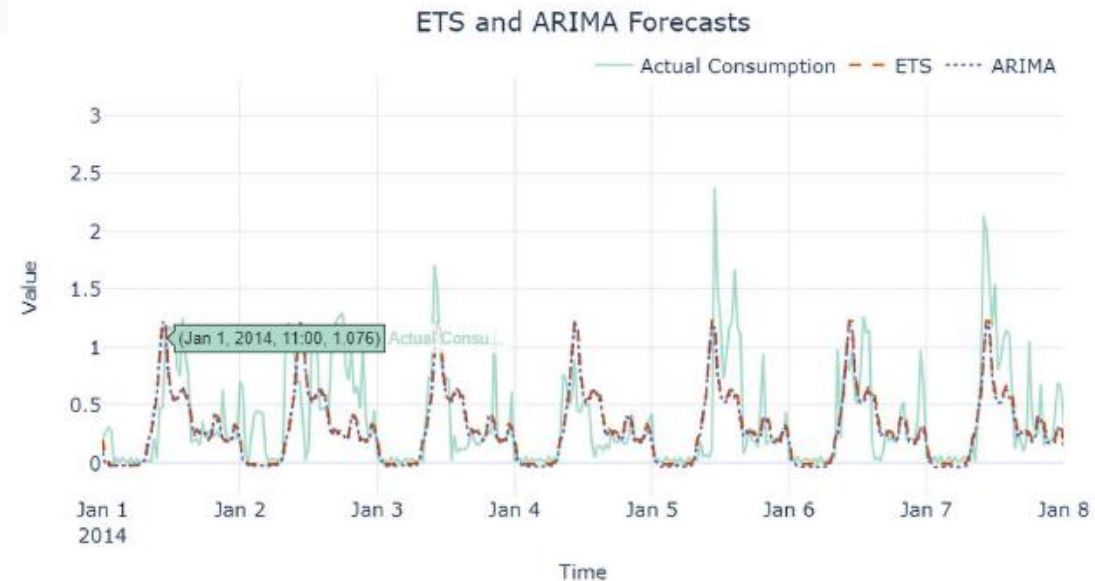


Figure 4.5: ETS and ARIMA forecasts

Model Selection

Which Model Should We Choose?

Competing models may have:

- Different complexity
- Different forecasting accuracy

Goal: Find the **simplest** model that explains the data well.

Benefits:

- Better interpretability
- Reduced overfitting
- Improved generalization

Akaike Information Criterion (AIC)

ARMA and other models are usually estimated with the **maximum-likelihood estimation (MLE)**.

One of the most commonly used model selection criteria for the maximum-likelihood method is the **Akaike information criterion (AIC)**,

$$AIC = 2k - 2\ln(L)$$

Where:

k = number of parameters

L = likelihood

Lower AIC indicates a better model,
which means it has **few parameters**, but also **high log-likelihood**.

Akaike Information Criterion (AIC)

For an ARIMA model, we could write more specifically:

$$AIC = 2(p + q) - 2l$$

We omitted the parameter d since it doesn't introduce additional estimations.

Bayesian Information Criterion (BIC)

The **Bayesian Information Criterion (BIC)** was proposed for model selection a few years later (1978), by Gideon Schwarz, and looks very much like AIC.

It additionally takes N , the number of samples in the dataset:

$$BIC = k \cdot \ln(N) - 2l,$$

Exponential Smoothing

Exponential smoothing is a technique for **smoothing** time-series data using an **exponential window function**, which can be used to **forecast** time-series with seasonality and trend.

Simple Exponential Smoothing (SES)

Exponential Smoothing Basic Idea

Recent observations are more important than older ones.

$$s_t = \alpha x_t + (1 - \alpha)s_{t-1}$$

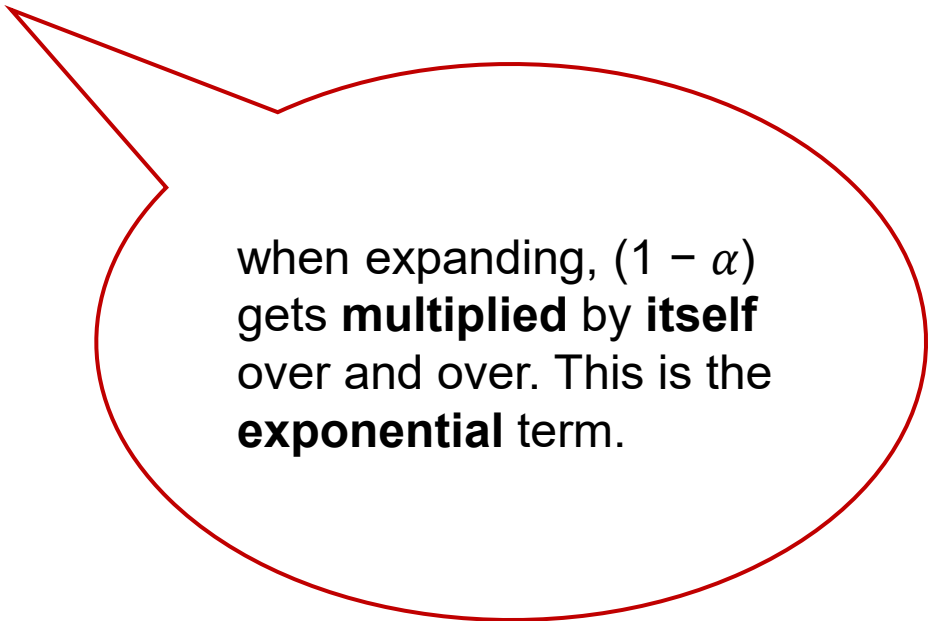
Where: α controls responsiveness

Characteristics:

- Effective for stationary series
- Smooths noise
- Fast and computationally efficient

Higher α – More responsive

Lower α – Smoother forecasts



when expanding, $(1 - \alpha)$ gets **multiplied** by **itself** over and over. This is the **exponential** term.

Theta Method

The **Theta method**, is another exponential smoothing method.

The Theta model can be understood as **simple exponential smoothing (SES)** with drift.

Competition-Winning Forecasting Method

The Theta method:

- Decomposes trend and short-term dynamics
- Uses exponential smoothing
- Performed exceptionally well in forecasting competitions

Key lesson:

Simple methods often outperform complex ones.

Theta Method

The **Theta method**, is another exponential smoothing method.

The Theta model can be understood as **simple exponential smoothing (SES)** with drift.

- T_t : estimated trend at time t
- c : intercept (starting level)
- b_0 : slope of the trend
- $t - 1$: time index
- ϵ_t : residual error

The trend component is forecast like this:

$$T_t = c + b_0(t - 1) + \epsilon_t$$

This is simply a **linear regression line over time**.

The formula for Theta is then taking this trend and adding it to the SES as a weighted sum:

$$\text{Theta: } \hat{X}_t = (1 - \alpha)T_t + \alpha \hat{X}_{t-1}$$

Theta forecast

```
theta_model = Theta(season_length =48, decomposition_type = 'additive' )
```

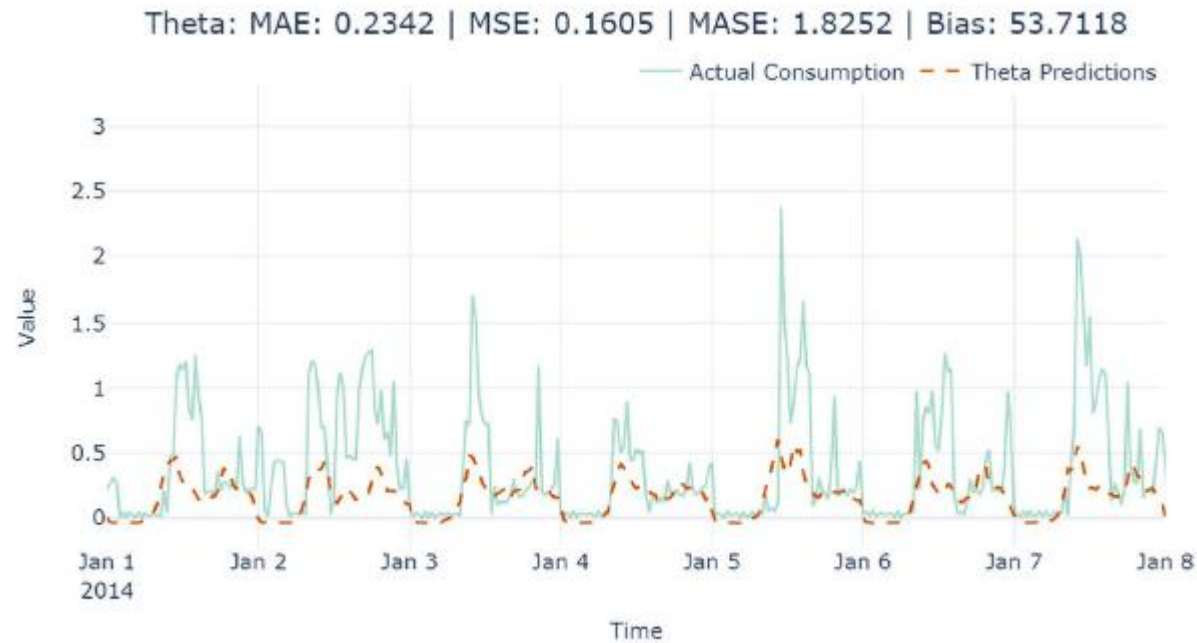


Figure 4.6: The Theta forecast

Holt-Winters Method

The Holt-Winters method is also called triple exponential smoothing because it applies exponential smoothing three times.

- An estimate of a level for each time **point**
- A **trend** component T
- **Seasonality** S with m seasons (the number of seasons in a year)

Suitable for seasonal forecasting problems.

Holt-Winters Method

With additive trend and seasonality, in mathematical terms, the Holtz-Winters forecast for a value x_{t+k} is defined as:

$$HW: x_{t+k} = L_t + kT_t + S_{t+k-m}$$

With multiplicative seasonality, we multiply by the seasonality:

$$HW: x_{t+k} = (L_t + kT_t) \cdot S_{t+k-m}$$

The level is updated as follows:

$$L_t = \alpha \frac{x_t}{S_{t-m}} + (1 - \alpha)(L_{t-1} + T_{t-1})$$

We are updating the current level based on a weighted average of two terms, with α being the weight between them. These two terms are the previous level and the de-seasonalized value of the series.

Holt-Winters Method

In this equation, we are de-seasonalizing the series by dividing by the seasonality:

$$\frac{x_t}{S_{t-m}}$$

And the previous trend component gets added up to the previous level like this:

$$L_{t-1} + T_{t-1}$$

The trend update is as follows (for additive trend):

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

Finally, the (multiplicative) seasonality update is as follows:

$$S_t = \gamma \frac{x_t}{L_t} + (1 - \gamma)S_{t-M}$$

Holt-Winters Method

```
from statsforecast.models import (SimpleExponentialSmoothing, Holt,  
HoltWinters, AutoETS)
```

```
exp_smooth = HoltWinters(error_type = 'A', season_length = 48)]
```

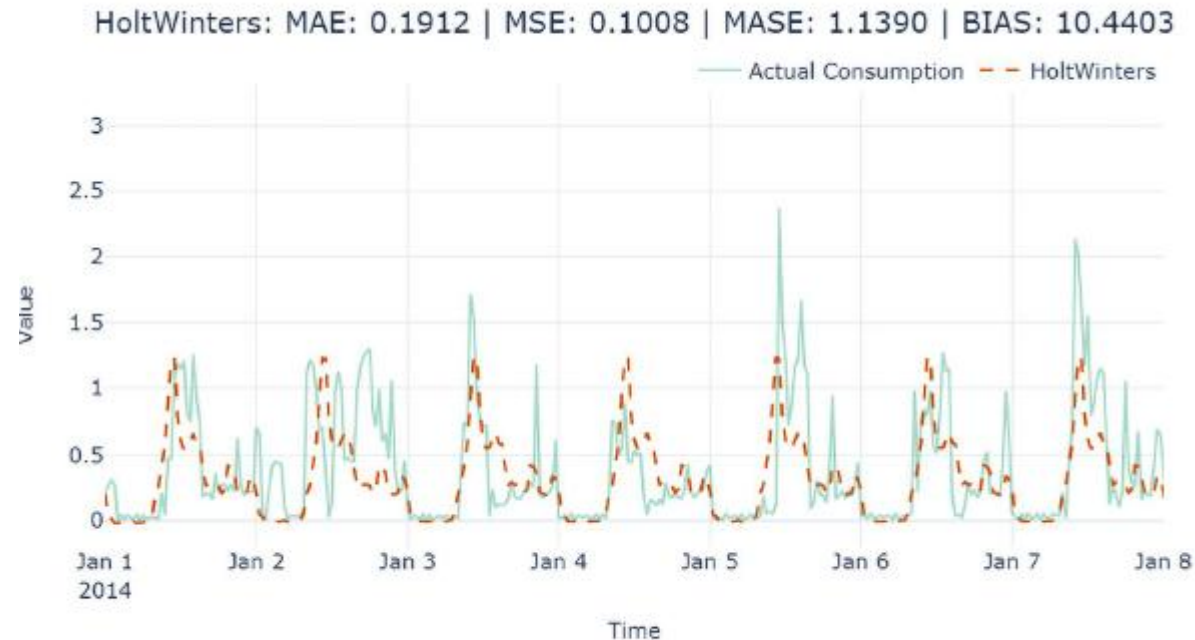


Figure 4.4: Exponential smoothing forecast

Additive vs Multiplicative Seasonality

Additive

Seasonal effect remains constant.

Multiplicative

Seasonal effect grows with trend magnitude.

Selection usually depends on visual inspection of data.

ARCH Models

Modeling Volatility

ARIMA models focus on predicting means.

ARCH models focus on predicting variance.

ARCH allows for time-varying (heteroscedastic) error terms by parametrizing higher-order dependence between returns observed at varying frequencies.

The standard deviation of the residual at time t is modeled depending on the residuals of the series at previous points:

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^q \alpha_i \epsilon_{t-i}^2$$

where q is the number of preceding time points the variance depends on.

The model ARCH(q) can be determined using least-squares.

GARCH Models

GARCH (generalized ARCH)

Improves ARCH by modeling:

- Previous errors
- Previous variances

Captures volatility clustering commonly observed in financial markets.

The main difference between GARCH and ARCH is that the residuals come from an ARCH model rather than from an autoregressive model, AR.

Vector Autoregression (VAR)

Vector autoregression models can track the **relationships between** several **variables** as they **change over time**. They can capture the **linear dependence** of a time-series on a vector of values that precedes the current timestamp, generalizing the AR model to **multivariate** time-series.

VAR(p) Model

$$x_t = c + B_1x_{t-1} + \cdots + B_px_{t-p} + \epsilon_t$$

VAR assumes that the error terms have a mean of 0 and that there is no serial correlation of error terms.

Popular Forecasting Libraries

- statsmodels
- pmdarima
- arch
- sktime
- darts

Most classical forecasting implementations rely on statsmodels.

Important Statsmodels Models

Common Classes

- AutoReg
- ARIMA
- ExponentialSmoothing
- SimpleExpSmoothing

Support forecasting, diagnostics, and statistical testing.

Statistical Tests

Useful tools:

- Augmented Dickey-Fuller (ADF)
- KPSS
- PACF
- ACF
- ARCH-LM test
- Ljung-Box test

Help validate assumptions before modeling.

Forecasting Example Workflow

1. Load data
2. Resample time series
3. Check stationarity
4. Analyze ACF/PACF
5. Select model order
6. Train model
7. Generate forecasts
8. Evaluate performance

Forecast Evaluation

If $f(x_t)$ is a prediction of the model for time step t , and the actual target value is y_t , intuitively, for a particular point, t , of our dataset, the forecast error (also prediction error or residual) is the difference between the actual values of the target and the values our model predicts:

$$e_t = y_t - f(x_t)$$

Common metrics:

Metric Name	Definition
Mean squared error	$\text{MSE} = \frac{1}{N} \sum_{t=1}^N e_t^2$
Mean absolute error	$\text{MAE} = \frac{1}{N} \sum_{t=1}^N e_t $
Root mean squared error	$\text{RMSE} = \sqrt{\text{MSE}}$

Goal:

Minimize forecasting error on unseen data.

More metrics for regression

Metric Name	Definition
Median absolute error	$\text{MdAE} = \text{median}(e _t)$
Mean absolute percentage error	$\text{MAPE} = \frac{1}{N} \sum_{t=1}^N \frac{ e_t }{ y_t }$
Symmetric mean absolute percentage error	$\text{SMAPE} = \frac{1}{N} \sum_{t=1}^N \frac{ e_t }{(y_t + f(x_t))/2}$
Normalized mean squared error	$\text{NMSE} = \frac{\text{MSE}}{\sigma^2}$

Metric Name	Definition
Root mean squared logarithmic error	$\text{RMSLE} = \sqrt{\frac{1}{N} \sum_{t=1}^N (\log(f(x_t) + 1) - \log(y_t + 1))^2}$